

Assignment – 5

Software Evolution, Emerging Paradigms & Green Computing

SIMATS School of Engineering | Department of Computer Science & Engineering

Name

CH. Chakradhar

Reg. no

192365043

Course Name

Software Engineering

Course Code

CSA1001

Question 1: Technical Debt and Management During Software Evolution

Prompt:

Explain technical debt. What are the major causes of technical debt, and how can organizations minimize it during software evolution? [10 Marks][cite: 9]

1. Concept and Metaphor of Technical Debt

Technical Debt (first conceptualized by Ward Cunningham in 1992) is a software engineering metaphor comparing expedient engineering compromises to financial debt. It describes the implied long-term cost incurred when software teams choose an easy, suboptimal, or hurried technical solution today to ship features rapidly, rather than implementing a robust, well-architected, and fully tested design.

Just like financial debt, technical debt compounds interest: every future modification, feature addition, or architectural expansion takes progressively longer and introduces greater bug risks because developers must navigate convoluted workarounds, tightly coupled codebases, and missing automated regression suites.

2. Major Causes of Technical Debt

- Commercial & Time-to-Market Pressure:

Accelerated release cycles force developers to bypass comprehensive architectural reviews, unit testing, and input validation to deliver features ahead of strict market deadlines.
- Inadequate Test Coverage & Lack of Automation:

Incomplete test suites allow silent defects and fragile regressions to propagate across the codebase, making future refactoring risky and unpredictable.
- Architectural Decay and Anti-Patterns:

Continuous feature extensions without refactoring degrade modular boundaries, resulting in "God classes", tight coupling, copy-paste programming, and monolithic dependencies.
- Outdated Dependencies & Documentation Decay:

Failing to upgrade frameworks, runtimes, and third-party libraries causes security vulnerabilities, deprecation warnings, and runtime inefficiencies to accumulate.
- Lack of Code Standardization & Knowledge Silos:

Inconsistent coding conventions, missing design documentation, and absence of peer review result in poorly understood, non-maintainable modules.

Quadrant Type	Deliberate (Intentional)	Inadvertent (Accidental)
Prudent	"We must ship this MVP now to validate product-market fit; we schedule a refactoring sprint immediately afterwards."	"Now that the system has evolved, we realize how the architecture should have been structured."

Quadrant Type	Deliberate (Intentional)	Inadvertent (Accidental)
Reckless	"We do not have time for unit tests or modular design; push code directly to production."	"The team lacked knowledge of design patterns or layered architecture and wrote monolithic code."

3. Strategies to Minimize Technical Debt During Software Evolution

- **Enforcing Static Code Analysis & Quality Gates:** Integrate tools like SonarQube, CodeClimate, and linters into CI/CD pipelines to block commits that violate cyclomatic complexity thresholds or introduce security vulnerabilities.
- **Dedicated Refactoring Sprints & Technical Debt Backlog:** Formalize debt tracking by documenting architectural flaws in Jira. Allocate 15–20% of every development sprint strictly to technical debt reduction and refactoring.
- **Test-Driven Development (TDD) & Automated Regression:** Build comprehensive unit, integration, and contract test suites to ensure developers can safely refactor components without fear of introducing regressions.
- **Domain-Driven Decomposition & Modular Boundaries:** Decompose monolithic systems into decoupled microservices or modular packages with strictly defined public APIs and contracts to contain architectural sprawl.
- **Continuous Architecture Reviews & Peer Audits:** Enforce rigorous Pull Request (PR) checklists requiring architectural approval from senior engineers before integrating complex changes.

Question 2: Impact of AI/ML and Edge Computing on Software Engineering

Prompt: Explain how AI/ML and Edge Computing are transforming modern software engineering. Provide one practical example for each. [10 Marks][cite: 9]

1. Transformation via Artificial Intelligence & Machine Learning (AI/ML)

Artificial Intelligence and Machine Learning have transformed software engineering from deterministic, rule-based systems into probabilistic, data-driven systems. This evolution impacts both **how software is built** (developer tooling) and **how systems operate** (adaptive runtime behavior):

- **Shift to Probabilistic Software Paradigms:** Traditional engineering specifies exact algorithms for every state. AI/ML systems learn decision boundaries directly from data distributions, requiring engineers to design for continuous retraining, data pipeline validation, model versioning (MLOps), and concept drift.
- **AI-Assisted Development Life Cycle:** LLM-powered coding assistants (e.g., GitHub Copilot) automate boilerplate generation, test synthesis, code explanation, and vulnerability detection, accelerating developer velocity and automated PR reviews.
- **Intelligent Predictive Operations (AIOps):** Runtime telemetry data is analyzed by machine learning models to detect real-time anomalies, predict server failure cascades, and orchestrate automated self-healing workflows.

Practical Real-World Example (AI/ML): *Intelligent Autonomous Fraud Detection in FinTech (e.g., Stripe Radar / PayPal).* Instead of hardcoding hundreds of brittle if-else rules to detect credit card fraud, machine learning models continuously analyze transaction metadata (device fingerprint, geolocation, velocity of requests, purchase patterns) in real time. The model computes a dynamic fraud score within milliseconds, blocking fraudulent transactions and adapting autonomously to evolving attack patterns without requiring code redeployments.

2. Transformation via Edge Computing

Edge Computing fundamentally shifts distributed software engineering by shifting computational workloads and data storage away from centralized hyperscale cloud data centers to nodes situated physically closer to end-user devices, IoT gateways, and telecom base stations:

- **Ultra-Low Latency & Bandwidth Conservation:** Eliminates round-trip transmission delays to distant cloud servers, enabling sub-millisecond response times critical for mission-critical real-time applications while drastically reducing bandwidth transit costs.
- **Architectural Decentralization & Partition Tolerance:** Software must be engineered for intermittent connectivity, local data caching, distributed consensus, and autonomous local operation when isolated from the central network.
- **Enhanced Privacy & Data Sovereignty:** Sensitive user telemetry, biometric scans, and telemetry data are processed locally on-premise or on device, satisfying strict privacy regulations (e.g., GDPR, HIPAA) without uploading raw data to the cloud.

Practical Real-World Example (Edge Computing): *Autonomous Vehicle Sensor Processing & Collision Avoidance (e.g., Tesla Full Self-Driving / Waymo).* An autonomous vehicle cannot depend on sending raw LIDAR, radar, and camera video streams to a remote AWS or Azure cloud server to detect pedestrians or brake for sudden obstacles, as network round trips (50–200ms) would cause fatal collisions. Instead, onboard high-performance edge

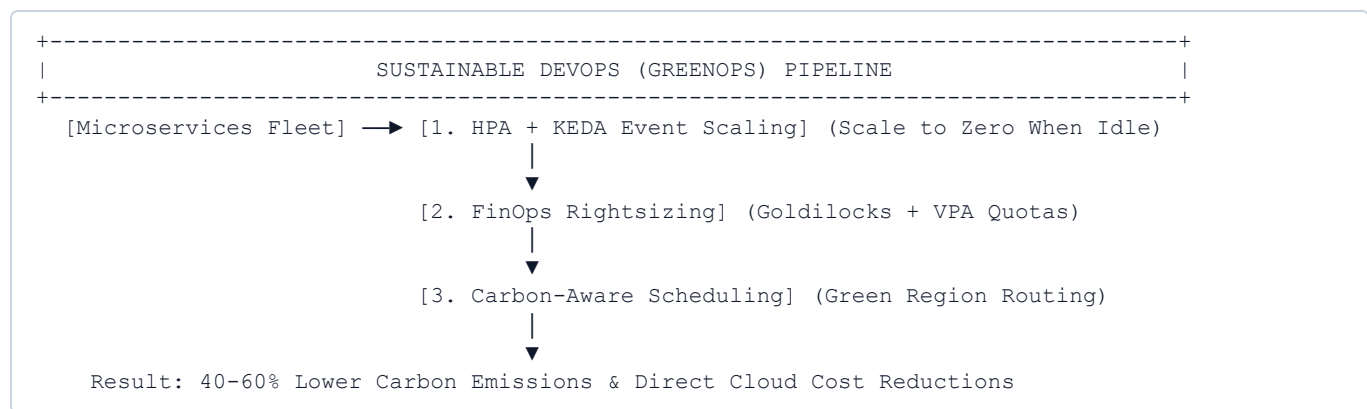
compute nodes process gigabytes of sensor telemetry per second locally with sub-5ms deterministic latency to make real-time steering and braking decisions, offloading only aggregated diagnostic metrics to the central cloud asynchronously.

Question 3: Case Study – Sustainable DevOps for Cloud Microservices

Prompt: A company deploys dozens of microservices in the cloud and notices that resource usage and energy consumption are increasing. Explain three sustainable DevOps practices that can reduce energy consumption and cloud cost. [5 Marks][cite: 9]

1. Incident Context & Green Cloud Engineering

Deploying dozens of independent microservices in the cloud often introduces hidden inefficiencies: over-provisioned CPU/memory limits, idle running containers during non-business hours, redundant inter-service network hops, and persistent orphan storage volumes[cite: 9]. Sustainable DevOps (GreenOps) focuses on measuring, optimizing, and reducing the computational carbon footprint while directly lowering operational cloud expenditure.



2. Three Concrete Sustainable DevOps Practices

Practice 1: Intelligent Dynamic Autoscaling and Scale-to-Zero Architecture (KEDA & HPA)

- **Technical Problem:** Microservices often remain provisioned at peak capacity 24/7, consuming continuous electrical power and cloud budget even during off-peak hours (e.g., overnight or weekends).
- **Sustainable Solution:** Implement Kubernetes Horizontal Pod Autoscaler (HPA) combined with Kubernetes Event-driven Autoscaling (KEDA). Non-critical microservices (e.g., report generators, batch processors, dev/test environments) are configured to scale down to zero replicas when request queues are empty. Workloads scale up only upon receiving ingress messages or events, cutting idle server energy waste to zero.

Practice 2: Continuous Resource Rightsizing and Elimination of Over-Provisioning

- **Technical Problem:** Software engineers routinely set container resource requests and limits excessively high (e.g., requesting 4 vCPUs and 8GB RAM for a service that uses 300m CPU and 512MB RAM) out of fear of out-of-memory (OOM) crashes, causing massive server underutilization.
- **Sustainable Solution:** Deploy autonomous container rightsizing tools such as Kubernetes Vertical Pod Autoscaler (VPA) and open-source profiling tools (e.g., Goldilocks, Kubecost). These tools analyze empirical CPU/memory telemetry and dynamically downsize container resource requests to realistic operating ranges, increasing container packing density per compute node and retiring redundant cloud VMs.

Practice 3: Carbon-Aware Scheduling and Graviton/ARM Architecture Migration

- **Technical Problem:** Legacy x86 CPU architectures consume significantly higher wattages per instruction, and continuous cloud workloads run in data centers powered by carbon-intensive fossil fuel grids.
- **Sustainable Solution:** Migrate containerized workloads to high-efficiency ARM-based processors (e.g., AWS Graviton3/4 or Ampere Altra), which deliver up to 60% higher performance-per-watt and 20% lower cost. Additionally, integrate carbon-aware workload orchestrators (such as the Green Software Foundation's Carbon-Aware SDK) to execute deferrable batch processing and CI pipelines during times and in regions where renewable solar/wind energy is abundant on the regional electrical grid.